

Vulkan Notes

VkImage, Render Pass & Framebuffer

Adam HAMOUCH

June 2026

1. VkImage — la brique de base

Une **VkImage** est un **buffer 2D de donnees GPU** — un tableau de pixels en memoire. Ce n'est pas forcément ce qui s'affiche a l'ecran. Toutes les cibles de rendu, textures et attachments sont des **VkImage**.

On n'accede jamais directement a une **VkImage** — on passe toujours par une **VkImageView** qui decrit comment l'interpreter (format, type, canaux, mipmaps).

Image layout : une image doit etre dans le bon *layout* selon son usage courant. Le GPU organise les pixels differemment en memoire selon ce qu'il va en faire.

Layout	Usage
UNDEFINED	Pas initialise, contenu inexploitable
COLOR_ATTACHMENT_OPTIMAL	Cible de rendu couleur (render target)
DEPTH_STENCIL_ATTACHMENT_OPTIMAL	Cible depth/stencil
SHADER_READ_ONLY_OPTIMAL	Lue dans un shader (texture, G-Buffer...)
PRESENT_SRC_KHR	Prête a etre presentee a l'ecran
TRANSFER_SRC/DST_OPTIMAL	Copie memoire (mipmaps, staging...)

Les transitions de layout sont gerees automatiquement par la render pass entre les subpasses, ou manuellement via des *pipeline barriers*.

2. Render Pass

La render pass est un **contrat** qui decrit la structure d'une passe de rendu — quelles images sont utilisees, dans quel etat elles entrent et dans quel etat elles sortent. Elle ne contient pas de donnees, de shaders ni de draw calls.

Attachments : les images impliquees dans la passe. Chaque attachment declare :

- Son format (RGBA8, Depth32...)
- **loadOp** : que faire en entrant — **CLEAR** (efface), **LOAD** (conserve), **DONT_CARE**
- **storeOp** : que faire en sortant — **STORE** (conserve), **DONT_CARE** (jette)
- Les layouts initial et final (transition automatique)

Subpasses : une render pass peut contenir plusieurs sous-etapes. Les subpasses peuvent lire les resultats de la precedente directement depuis les attachments sans passer par la memoire principale — optimisation majeure sur GPU mobile. Pour le rendu de base, une seule subpass suffit.

Pour le moteur : le deferred rendering utilisera plusieurs subpasses ou render passes en sequence — G-Buffer, lighting, post-process. Chaque passe a son propre loadOp/storeOp adapte.

Passe	loadOp	storeOp
Color (nouvelle frame)	CLEAR	STORE
Depth (forward)	CLEAR	DONT_CARE
Shadow map	CLEAR	STORE
G-Buffer (deferred)	CLEAR	STORE
Accumulation multi-pass	LOAD	STORE

La render pass est creee **independamment** du framebuffer. Elle declare des slots abstraits. Le framebuffer les remplit avec les vraies images. La render pass peut etre partagee entre plusieurs framebuffers de meme structure.

3. Framebuffer

Le framebuffer fait le **lien concret** entre une render pass et les images sur lesquelles le GPU va reellement ecrire. C'est une collection de `VkImageView` — une par slot declare dans la render pass.

Pour du rendu basique, on a toujours au minimum deux images :

- **Color image** (swap chain image) : le GPU y ecrit les couleurs. C'est cette image qui sera presentee a l'ecran.
- **Depth image** : le GPU y ecrit les valeurs de profondeur pour le depth test. Partagee entre toutes les frames.

On cree **un framebuffer par image de la swap chain** (2 ou 3 selon le mode de presentation). Le bon framebuffer est selectionne au moment du rendu de chaque frame.

Swap chain images vs depth image : les images de la swap chain sont differentes a chaque framebuffer. La depth image elle est partagee entre tous — le depth buffer est efface (CLEAR) au debut de chaque frame de toute facon.

4. Les quatre acteurs d'une passe de rendu

Objet	Role
<code>VkRenderPass</code>	Le plan : structure abstraite, formats, loadOp/storeOp
<code>VkFramebuffer</code>	Les vraies images qui remplissent le plan
<code>VkPipeline</code>	Les shaders et la configuration GPU pour cette passe
<code>VkCommandBuffer</code>	Les draw calls enregistres

La render pass est creee une fois et reutilisee. Le framebuffer est recree si la fenetre est redimensionnee (la swap chain change de taille). Les images de la swap chain ne changent jamais de contenu entre deux frames — on reecrit entierement dedans a chaque frame via les command buffers.

Ordre de destruction : Framebuffers → Pipeline → Pipeline Layout → Render Pass → Image Views → Depth Image → Swap Chain → Surface → Logical Device → Instance.